Package 'surfscape'

October 13, 2026

Type Package

Title Composition and Configuration in Continuous Landscape Surfaces

Version 0.3.0

Description Tests how continuous landscape metrics, such as surface, texture, autocorrelation, entropy and lacunarity metrics computed on vegetation indices or other continuous surfaces, respond to changes in the values present in a landscape window (composition) and to changes in their spatial arrangement (configuration). A battery of controlled changes (spatial permutation, uniform shift, scaling, rank-preserving change of distribution shape and landscape-wide rearrangement) classifies any metric, including implementations from other packages, as compositional, configurational or mixed. Continuous neutral landscapes are simulated by circulant embedding of Gaussian random fields. The change in a moving-window metric between two dates is decomposed exactly into a landscape-wide compositional, a local compositional and a configurational component using rank-transfer counterfactuals and the two-factor Shapley value. Built on 'terra' with a 'Rcpp' window engine.

License GPL-3

Encoding UTF-8

Depends R (>= 4.1)

Imports terra (>= 1.7), Rcpp (>= 1.0.10), stats, graphics

LinkingTo Rcpp

Suggests testthat (>= 3.0.0), knitr, rmarkdown

Config/testthat/edition 3

URL https://github.com/ehsanrahimi666/surfscape

BugReports https://github.com/ehsanrahimi666/surfscape/issues

NeedsCompilation yes

VignetteBuilder knitr

Config/roxygen2/version 8.1.1

Contents

ss_benchmark 2
ss_benchmark_combine 5
ss_decompose 6
ss_decompose_landscape 8
ss_extract 9
ss_metrics 10
ss_metric_list 11
ss_perturb 12
ss_rank_transfer 14
ss_scale_profile 15
ss_simulate 16

Index **18**

`ss_benchmark` *Benchmark landscape metrics against controlled changes of composition and configuration*

Description

Tests how a continuous landscape metric responds to changes in the values present (composition) and to changes in their spatial arrangement (configuration). Landscape windows are sampled from a surface, each test is applied, and the metric is evaluated on the same windows before and after the change. The same design is applied to the built-in metrics and to any metric function supplied by the user, including implementations from other packages, so that they can be compared on identical windows.

Usage

```
ss_benchmark(
  x,
  metrics = NULL,
  fun = NULL,
  w = 15,
  n = 100,
  nperm = 39,
  tests = c("permute", "shift", "scale", "reshape"),
  shift = 0.5,
  scale = 1.5,
  skew = 2,
  target = NULL,
  arrangement = NULL,
  sim_range = NULL,
  protocol = c("fixed", "own"),
  cells = NULL,
  seed = NULL,
```

```
  tol = 1e-09,
  alpha = 0.05,
  fun_input = c("matrix", "raster"),
  nbins = 32L,
  nlevels = 16L,
  lacbox = 3L,
  details = FALSE
)
```

Arguments

x	A single-layer 'SpatRaster'.
metrics	Built-in metric names, see [ss_metric_list()]. Defaults to all of them when 'fun' is not supplied.
fun	Alternatively, a function that takes one landscape window and returns a named numeric vector of metric values, the same length for every window. If 'fun' has an argument called 'range', it receives the value range used for binning under the chosen 'protocol'; if it has an argument called 'surface', it receives the values that define that range (all values of both surfaces compared under '"fixed"', of the surface the window is taken from under '"own"'), for binning schemes based on the value distribution, such as equal-probability quantization.
w	Window size in cells, a single odd integer or 'c(nrow, ncol)', or '"landscape"' to treat the whole surface as a single window.
n	Number of windows sampled at random among the complete windows (no missing values) of 'x'. Ignored when 'cells' is supplied or 'w = "landscape"'.
nperm	Number of random spatial permutations per window for the '"permute"' test.
tests	Any of '"permute"', '"shift"', '"scale"', '"reshape"', '"rearrange"' and '"target"'; see Details.
shift, scale, skew	Sizes of the '"shift"' (in standard deviations of 'x'), '"scale"' (stretch factor) and '"reshape"' (target skewness) tests.
target	A 'SpatRaster' with the geometry of 'x' whose value distribution is imposed on 'x' by the '"target"' test, for example the same landscape at a second date.
arrangement	A 'SpatRaster' with the geometry of 'x' whose spatial ranks receive the values of 'x' in the '"rearrange"' test. If 'NULL', a surface is simulated with [ss_simulate()] using practical range 'sim_range' (default: the window size).
sim_range	See 'arrangement'.
protocol	How binned metrics (entropy, Rao's Q, lacunarity and grey level co-occurrence textures) set their value range when the surface changes. '"fixed"': both surfaces of a comparison share their joint value range, as required when values are compared between dates. '"own"': each surface uses its own value range.
cells	Optional cell numbers of the window centres, for example the 'cells' element of a previous result, so that different metric sets can be benchmarked on the same windows.

`seed` Optional integer seed. The random-number state of the session is restored on exit.

`tol` Relative numerical tolerance. A test leaves a metric invariant when no window changes by more than ‘tol‘ times the largest absolute value of the metric over all evaluated windows (observed, permuted and changed by any test), which absorbs floating-point rounding only.

`alpha` Significance level of the per-window permutation test.

`fun_input` Whether ‘fun‘ receives each window as a numeric ‘"matrix"‘ or as a ‘"raster"‘ (‘SpatRaster‘ with the window’s geometry and coordinate reference system).

`nbins`, `nlevels`, `lacbox` Settings of the built-in metrics, see [ss_metrics()].

`details` Logical. If ‘TRUE‘, the per-window values are returned.

Details

The tests separate the two properties of a landscape:

permute the values inside each window are randomly rearranged. Composition is unchanged and configuration is destroyed.

shift a constant is added to every cell.

scale values are stretched about their mean, which changes their spread but not their mean.

reshape the shape of the value distribution changes while the mean, the spread and the spatial ordering of the values are kept.

target the value distribution of ‘target‘ is imposed while the spatial ordering of ‘x‘ is kept.

rearrange the values of the whole surface are moved to new locations while the landscape-wide value distribution is kept. Inside a window this changes both which values are present and how they are arranged, so it is reported but not used for classification.

‘shift‘, ‘scale‘, ‘reshape‘ and ‘target‘ change composition and keep the spatial ordering of values. A metric is classified as **compositional** when it is invariant to ‘permute‘ and responds to at least one composition test or to ‘rearrange‘ (a metric that is invariant to ‘permute‘ can only respond to ‘rearrange‘ through the change in the values present), **configurational** when it responds to ‘permute‘ and is invariant to every composition test, **mixed** when it responds to ‘permute‘ and to at least one composition test, and **unresponsive** when it is invariant to all tests.

The response to a test is the mean absolute change of the metric divided by its standard deviation among the sampled windows, which expresses the change on the scale of the metric’s natural variation across the landscape. In the permutation test each window is tested separately, two-sided, by comparing its observed value with its ‘nperm‘ permuted values; with ‘nperm = 39‘ and ‘alpha = 0.05‘ a window is significant only when its observed value lies beyond all permuted values in one direction (Hope 1968). Values equal within the numerical tolerance count as ties. Fewer than ‘2 / alpha - 1‘ permutations cannot give a significant result, and a warning is issued.

The tests differ in size: the permutation removes all spatial structure inside the window, whereas the composition tests are moderate changes. Responses are therefore comparable among metrics for the same test, not between tests; the classification uses only whether a metric is invariant.

Value

An object of class ‘ss_benchmark‘, a list with

summary one row per metric: ‘category‘, ‘configuration_response‘ (response to ‘"permute"‘), ‘composition_response‘ (largest response among the composition tests), ‘signif‘ (proportion of windows in which the observed arrangement differs significantly from random arrangements) and, for each test, whether the metric was invariant. Responses to tests to which the metric is invariant are reported as 0; the unrounded values are in ‘responses‘.

responses one row per metric and test with ‘n‘ (windows), ‘invariant‘, ‘max_abs‘ and ‘mean_abs‘ (largest and mean absolute change), ‘mean_change‘ (mean signed change), ‘response‘ (‘mean_abs‘ divided by the standard deviation of the metric among windows; ‘NA‘ when the metric does not vary among windows or only one window is evaluated), and, for ‘"permute"‘, ‘signif‘ and ‘ses‘ (mean standardised effect size, observed minus permutation mean over permutation standard deviation).

cells the window centres.

settings the settings used.

values per-window values, when ‘details = TRUE‘.

References

Hope ACA (1968) A simplified Monte Carlo significance test procedure. *Journal of the Royal Statistical Society B* 30:582-598.

See Also

[ss_perturb()] for the tests themselves, [ss_simulate()] for landscapes with known structure.

Examples

```
z <- ss_simulate(80, 80, range = 10, seed = 1)
b <- ss_benchmark(z, metrics = c("sq", "sent", "sr1", "gcon"), w = 9,
                  n = 30, nperm = 39, seed = 1)
b

# any metric function, evaluated on the same windows
f <- function(m) c(sd = stats::sd(as.vector(m)),
                   lag1 = mean((m[, -ncol(m)] - m[, -1])^2))
ss_benchmark(z, fun = f, w = 9, nperm = 39, cells = b$cells, seed = 1)
```

`ss_benchmark_combine` *Combine benchmark results computed on separate sets of windows*

Description

Combines [ss_benchmark()] results obtained with identical settings on different windows, for example windows processed in parallel or windows sampled from replicate landscapes, and recomputes all statistics on the pooled windows.

Usage

```
ss_benchmark_combine(...)
```

Arguments

`...`	'ss_benchmark' objects computed with 'details = TRUE', or a single list of them.

Value

An object of class 'ss_benchmark' with 'details' included.

Examples

```
z1 <- ss_simulate(60, 60, range = 8, seed = 1)
z2 <- ss_simulate(60, 60, range = 8, seed = 2)
b1 <- ss_benchmark(z1, metrics = c("sq", "sr1"), w = 9, n = 20,
                   seed = 1, details = TRUE)
b2 <- ss_benchmark(z2, metrics = c("sq", "sr1"), w = 9, n = 20,
                   seed = 2, details = TRUE)
ss_benchmark_combine(b1, b2)
```

---

ss_decompose *Decompose change in a continuous landscape surface*

Description

Splits the change in a moving-window landscape metric between two dates into three additive components: a change in the landscape-wide value distribution, a redistribution of values among landscape windows, and a rearrangement of values within windows. The three components sum to the observed change at every cell, up to floating-point rounding.

Usage

```
ss_decompose(
  z1,
  z2,
  metrics = c("sq", "sr1"),
  w = 7,
  components = c("total", "global", "local", "config"),
  range = NULL,
  nbins = 32L,
  nlevels = 16L,
  lacbox = 3L,
  filename = "",
  overwrite = FALSE,
  chunk_rows = NULL,
  wopt = list()
)
```

Arguments

`z1`, `z2`	Single-layer ‘SpatRaster‘ objects for the two dates, with identical geometry. Cells missing in either date are dropped from both.
`metrics`	Character vector of metric names, see [ss_metric_list()].
`w`	Window size in cells: an odd integer or ‘c(nrow, ncol)‘.
`components`	Which components to return. Any subset of ‘"total"‘, ‘"global"‘, ‘"local"‘, ‘"config"‘.
`range`	Value range shared by both dates, used for binning. Defaults to the range of the two dates combined, which is the correct choice for a temporal comparison.
`nbins`, `nlevels`, `lacbox`	Metric settings, see [ss_metrics()].
`filename`, `overwrite`, `chunk_rows`, `wopt`	Output and blocking control.

Value

A ‘SpatRaster‘ with one layer per metric and component, named ‘<metric>_<component>‘.

The decomposition

For every window, the change in a metric between the two dates is split into three parts that add up to the total change:

global the part due to the change in the landscape-wide value distribution;

local the part due to values moving between windows, so that a window contains a different set of values even when the landscape-wide distribution is unchanged;

config the part due to the rearrangement of values inside the window.

The components are obtained from counterfactual surfaces built by rank transfer (see [ss_rank_transfer()]), which keeps the spatial ordering of values of one date and gives it the values of the other date. Inside each window, rank transfer between the two dated windows separates the change due to the window's values (marginal part) from the change due to their arrangement (‘config‘). Over the whole landscape, rank transfer between the two dated surfaces gives the change due to the landscape-wide distribution (‘global‘), and ‘local‘ is the marginal part minus ‘global‘. Each counterfactual comparison is averaged over the two possible orderings of the dates (the two-factor Shapley value), so the result does not depend on which date is taken first; where composition and configuration change together, their interaction is shared equally between the components. The formulas are given in the package vignette.

Two limiting cases follow exactly from this construction. If the second date is a strictly increasing transformation of the first, the spatial ordering of values is unchanged everywhere and the ‘config‘ and ‘local‘ components are zero. If the two dates contain the same values in a different arrangement, the landscape-wide distribution is unchanged and the ‘global‘ component is zero, while the ‘local‘ and ‘config‘ components are generally not zero because values have moved between and within windows.

For a metric that depends only on the values present in the window (a compositional metric, see [ss_benchmark()]) the ‘config‘ component is zero at every cell and at every window size.

See Also

[ss_benchmark()] to classify a metric before using it, [ss_scale_profile()] to run the decomposition across scales.

Examples

```
set.seed(1)
z1 <- terra::rast(nrows = 40, ncols = 40)
terra::values(z1) <- stats::rnorm(1600)
z1 <- terra::focal(z1, 5, "mean")
z2 <- terra::rast(z1)
terra::values(z2) <- stats::rnorm(1600)
z2 <- terra::focal(z2, 9, "mean")
d <- ss_decompose(z1, z2, metrics = c("sq", "sr1"), w = 7)
names(d)
```

---

ss_decompose_landscape

Decompose change in landscape-level metrics

Description

Splits the change in metrics computed over the whole surface between two dates into a compositional component, due to the change in the value distribution, and a configurational component, due to the change in the spatial arrangement of values. At this level the landscape window is the whole surface, so the local component of [ss_decompose()] does not arise.

Usage

```
ss_decompose_landscape(
  z1,
  z2,
  metrics = NULL,
  fun = NULL,
  range = NULL,
  nbins = 32L,
  nlevels = 16L,
  lacbox = 3L
)
```

Arguments

`z1`, `z2`	Single-layer 'SpatRaster' objects for the two dates, with identical geometry and no missing values.
`metrics`	Built-in metric names, see [ss_metric_list()].
`fun`	Alternatively, a function that takes a 'SpatRaster' and returns a named numeric vector, for metrics implemented elsewhere.

`range`, `nbins`, `nlevels`, `lacbox`
: Settings of the built-in metrics, see [ss_metrics()]. ‘range‘ defaults to the joint range of both dates.

Details

The components are averages over the two orderings of the dates of the differences between the observed surfaces and the rank-transfer counterfactuals (one date's arrangement with the other date's values), as in [ss_decompose()]. If the second date is a strictly increasing transformation of the first, the configurational component is zero; if both dates contain the same values, the compositional component is zero.

Value

A ‘data.frame‘ with one row per metric: the values at both dates (‘value1‘, ‘value2‘), ‘total‘ change, ‘composition‘ and ‘configuration‘ components (which sum to ‘total‘), and ‘share_config‘, the absolute configurational component divided by the sum of the absolute values of both components.

Examples

```
z1 <- ss_simulate(60, 60, range = 6, seed = 1)
z2 <- ss_simulate(60, 60, range = 12, seed = 2)
ss_decompose_landscape(z1, z2, metrics = c("sq", "sr1", "gcon"))
ss_decompose_landscape(z1, z2, fun = function(r)
  c(moran = terra::autocor(r, method = "moran", global = TRUE)))
```

---

ss_extract *Extract decomposed change around point locations at several radii*

Description

Summarises the output of [ss_decompose()] in circular buffers around field sites, at several radii, and returns a tidy long table ready for multi-scale modelling.

Usage

```
ss_extract(x, pts, radii, fun = mean, coords = c("x", "y"), id = NULL)
```

Arguments

`x`	A ‘SpatRaster‘, typically the output of [ss_decompose()].
`pts`	An ‘sf‘ object, a ‘SpatVector‘ of points, a two-column matrix, or a ‘data.frame‘ with coordinate columns.
`radii`	Numeric vector of buffer radii in map units.
`fun`	Summary function applied within each buffer.
`coords`	Column names holding x and y when ‘pts‘ is a ‘data.frame‘.
`id`	Optional column name in ‘pts‘ to carry through as a site identifier.

Value

A long ‘data.frame‘ with columns ‘site‘, ‘radius‘, ‘layer‘, ‘value‘.

Buffer geometry

Buffers are circular and defined by radius, so buffer area scales with ‘radius^2‘. When comparing extraction shapes rather than sizes, match on area rather than on a linear dimension, otherwise any apparent effect of shape is confounded with a threefold difference in the area sampled.

Examples

```
set.seed(1)
z1 <- terra::rast(nrows = 50, ncols = 50, xmin = 0, xmax = 500,
                  ymin = 0, ymax = 500)
terra::values(z1) <- stats::rnorm(2500)
z2 <- terra::rast(z1); terra::values(z2) <- stats::rnorm(2500)
d <- ss_decompose(z1, z2, metrics = "sr1", w = 5)
pts <- data.frame(x = c(100, 300), y = c(100, 400))
ss_extract(d, pts, radii = c(50, 100))
```

---

ss_metrics *Moving-window metrics for continuous landscape surfaces*

Description

Computes continuous landscape metrics in a moving window. All metrics are computed from the same square landscape window around each cell, so that metrics that depend on the arrangement of values (lag-1 autocorrelation, slope, lacunarity, grey-level co-occurrence textures) and metrics that depend only on the values present (amplitude and information-theoretic metrics) describe exactly the same area.

Usage

```
ss_metrics(
  x,
  metrics = c("sq", "ssk", "sr1", "gcon"),
  w = 9,
  range = NULL,
  nbins = 32L,
  nlevels = 16L,
  lacbox = 3L,
  filename = "",
  overwrite = FALSE,
  chunk_rows = NULL,
  wopt = list()
)
```

Arguments

`x`	A single-layer 'SpatRaster', or anything [terra::rast()] accepts.
`metrics`	Character vector of metric names; see [ss_metric_list()].
`w`	Window size: a single odd integer, or 'c(nrow, ncol)', in cells.
`range`	Numeric length-2 value range used to bin 'sent', 'srao' and the grey levels of the GLCM metrics. Defaults to the range of 'x'. **Supply the same 'range' for both dates when comparing metrics through time**, otherwise the binning differs between dates and the comparison is not meaningful; [ss_decompose()] does this automatically.
`nbins`	Number of bins for 'sent' and 'srao'.
`nlevels`	Number of grey levels for the GLCM metrics.
`lacbox`	Gliding-box side length for 'slac'.
`filename`, `overwrite`, `chunk_rows`, `wopt`	Output and blocking options.

Value

A 'SpatRaster' with one layer per requested metric, in the order given by 'metrics'.

Relationship to other packages

geodiv implements a wider set of surface metrics, including the bearing-area, spectral and correlation-length families, **GLCMTextures** implements grey-level co-occurrence textures with more options, and [terra::autocor()] computes Moran's I and Geary's C. These should be preferred when those metrics are the goal, and any of them can be tested with [ss_benchmark()]. The metrics are reimplemented here only because [ss_decompose()] evaluates them on artificial windows created during the computation (for example, the values of one date in the arrangement of the other), which are not available as raster layers that could be passed to another package.

Examples

```
f <- system.file("extdata", "ndvi_2013.tif", package = "surfscape")
if (nzchar(f)) {
  r <- terra::rast(f)
  m <- ss_metrics(r, c("sq", "sr1", "gcon"), w = 9)
  names(m)
}
```

`ss_metric_list` *Catalogue of continuous landscape metrics*

Description

Lists the metrics computed by the **surfscape** window kernel with two properties that follow from their definitions: whether a metric depends on the positions of values inside the landscape window ('arrangement'), and whether it bins values on a value range ('binned'), which makes its response to changes in composition depend on how that range is set.

Usage

```
ss_metric_list()
```

Details

A metric with ‘arrangement = FALSE‘ is a function of the set of values in the window only: rearranging the values inside the window leaves it unchanged, so it describes composition and cannot respond to configuration. This holds for the amplitude metrics of surface metrology (McGarigal et al. 2009) and for the information-theoretic measures, as noted by Frazier (2016) and Kedron et al. (2018). A metric with ‘arrangement = TRUE‘ can respond to configuration, but that does not make it insensitive to composition; [ss_benchmark()] measures both responses for any metric.

Value

A ‘data.frame‘ with columns ‘metric‘, ‘id‘, ‘family‘, ‘arrangement‘, ‘binned‘ and ‘definition‘.

References

McGarigal K, Tagil S, Cushman SA (2009) Surface metrics: an alternative to patch metrics for the quantification of landscape structure. *Landscape Ecology* 24:433-450.

Frazier AE (2016) Surface metrics: scaling relationships and downscaling behavior. *Landscape Ecology* 31:351-363.

Kedron PJ, Frazier AE, Ovando-Montejo GA, Wang J (2018) Surface metrics for landscape ecology: a comparison of landscape models across ecoregions and scales. *Landscape Ecology* 33:1489-1504.

Haralick RM, Shanmugam K, Dinstein I (1973) Textural features for image classification. *IEEE Transactions on Systems, Man, and Cybernetics* 3:610-621.

Allain C, Cloitre M (1991) Characterizing the lacunarity of random and deterministic fractal sets. *Physical Review A* 44:3552-3558.

Plotnick RE, Gardner RH, Hargrove WW, Prestegaard K, Perlmutter M (1996) Lacunarity analysis: a general technique for the analysis of spatial patterns. *Physical Review E* 53:5461-5468.

Examples

```
ss_metric_list()
subset(ss_metric_list(), arrangement)
```

---

ss_perturb *Controlled changes of composition or configuration*

Description

Applies one controlled change to a continuous surface. Each change alters either the values present (composition) while keeping their spatial ordering, or the spatial arrangement while keeping the values present. These are the tests used by [ss_benchmark()], exposed so that they can be inspected, mapped or combined.

Usage

```
ss_perturb(
  x,
  type = c("shift", "scale", "reshape", "permute", "rearrange", "target"),
  amount = NULL,
  target = NULL,
  arrangement = NULL,
  sim_range = NULL,
  seed = NULL
)
```

Arguments

x	A single-layer ‘SpatRaster‘.
type	The change to apply: **‘"shift"‘** adds a constant, ‘amount‘ standard deviations of ‘x‘, to every cell (composition change; arrangement unchanged). **‘"scale"‘** stretches values about the mean by the factor ‘amount‘; the mean is unchanged and the spread changes (composition change; arrangement unchanged). **‘"reshape"‘** a rank-preserving change of distribution shape: the values are replaced by those of a gamma distribution with skewness ‘amount‘ (a normal distribution for ‘amount = 0‘) rescaled to the mean and standard deviation of ‘x‘. Mean, spread and the spatial ordering of values are unchanged (composition change; arrangement unchanged). **‘"permute"‘** randomly permutes all values across the surface (configuration change; values unchanged). **‘"rearrange"‘** reassigns the values of ‘x‘ to the spatial ranks of ‘arrangement‘, or of a simulated surface with practical range ‘sim_range‘ (configuration change at the extent of ‘x‘; values unchanged). **‘"target"‘** keeps the spatial ranks of ‘x‘ and gives it the value distribution of ‘target‘ (composition change; arrangement unchanged).
amount	Size of the change for ‘"shift"‘ (default 0.5), ‘"scale"‘ (default 1.5) and ‘"reshape"‘ (default 2).
target	A ‘SpatRaster‘ with the geometry of ‘x‘, for ‘"target"‘.
arrangement	A ‘SpatRaster‘ with the geometry of ‘x‘, for ‘"rearrange"‘.
sim_range	Practical range, in cells, of the surface simulated with [ss_simulate()] for ‘"rearrange"‘ when ‘arrangement‘ is not supplied.
seed	Optional integer seed for ‘"permute"‘ and ‘"rearrange"‘. The random-number state of the session is restored on exit.

Details

Ties in ‘x‘ are kept as ties by ‘"reshape"‘, so the transformation is a non-decreasing function of the original values and the spatial ordering of values inside any window is unchanged.

Value

A ‘SpatRaster‘ with the geometry of ‘x‘.

Examples

```
z <- ss_simulate(60, 60, range = 8, seed = 1)
zs <- ss_perturb(z, "shift")
zr <- ss_perturb(z, "reshape", amount = 2)
cor(terra::values(z)[, 1], terra::values(zr)[, 1], method = "spearman")
zp <- ss_perturb(z, "rearrange", sim_range = 20, seed = 2)
all.equal(sort(terra::values(z)[, 1]), sort(terra::values(zp)[, 1]))
```

---

ss_rank_transfer *Global rank transfer*

Description

Builds a counterfactual surface that keeps the spatial rank ordering of ‘arrangement‘ and takes the value distribution of ‘values‘. Only cells that are non-missing in both inputs are used.

Usage

```
ss_rank_transfer(arrangement, values)
```

Arguments

`arrangement, values`
: Single-layer ‘SpatRaster‘ objects with identical geometry.

Value

A ‘SpatRaster‘.

Examples

```
r1 <- terra::rast(nrows = 20, ncols = 20)
r2 <- terra::rast(nrows = 20, ncols = 20)
set.seed(1)
terra::values(r1) <- stats::rnorm(400)
terra::values(r2) <- stats::rnorm(400, 2)
ss_rank_transfer(r2, r1)
```

`ss_scale_profile` *Scale profile of decomposed landscape change*

Description

Runs [ss_decompose()] over a series of window sizes and summarises how the three components of change vary with the size of the landscape window.

Usage

```
ss_scale_profile(
  z1,
  z2,
  metrics = c("sq", "sr1"),
  w_seq = c(3, 7, 15, 31),
  range = NULL,
  nbins = 32L,
  nlevels = 16L,
  lacbox = 3L,
  rasters = FALSE,
  progress = TRUE
)
```

Arguments

`z1`, `z2`	Single-layer 'SpatRaster' objects for the two dates, with identical geometry. Cells missing in either date are dropped from both.
`metrics`	Character vector of metric names, see [ss_metric_list()].
`w_seq`	Vector of odd window sizes in cells.
`range`	Value range shared by both dates, used for binning. Defaults to the range of the two dates combined, which is the correct choice for a temporal comparison.
`nbins`, `nlevels`, `lacbox`	Metric settings, see [ss_metrics()].
`rasters`	Logical. If 'TRUE' the decomposition rasters for each scale are returned alongside the summary.
`progress`	Logical, print progress.

Value

A 'data.frame' with one row per window size and metric: 'n' (cells), the mean absolute value of the total change and of each component ('mad_*'), 'share_config', the mean absolute configurational component divided by the sum of the mean absolute values of the three components, the correlations across cells between the compositional (global plus local) and configurational components ('r_comp_config') and between the total change and the configurational component ('r_total_config'), and 'sign_disagree', the proportion of cells with a non-zero configurational component whose sign differs from that of the total change. Correlations are 'NA' when a component

is constant, and ‘sign_disagree‘ is ‘NA‘ when the configurational component is zero everywhere. When ‘rasters = TRUE‘, a list with elements ‘summary‘ and ‘rasters‘.

Examples

```
set.seed(1)
z1 <- terra::rast(nrows = 60, ncols = 60)
terra::values(z1) <- stats::rnorm(3600)
z1 <- terra::focal(z1, 5, "mean")
z2 <- terra::rast(z1)
terra::values(z2) <- stats::rnorm(3600)
z2 <- terra::focal(z2, 9, "mean")
ss_scale_profile(z1, z2, metrics = "sr1", w_seq = c(5, 9))
```

---

ss_simulate *Simulate a continuous neutral landscape*

Description

Generates a stationary Gaussian random field on a regular grid by circulant embedding. Such fields are the continuous counterpart of neutral landscape models: their spatial structure is set by a single parameter, the practical range, and their value distribution can be replaced by any other through rank transfer without altering the spatial arrangement. They provide landscapes with known properties for controlled tests of landscape metrics.

Usage

```
ss_simulate(
  nrow,
  ncol,
  range,
  model = c("exponential", "gaussian"),
  values = NULL,
  res = 1,
  crs = "local",
  seed = NULL
)
```

Arguments

nrow, ncol	Grid size in cells.
range	Practical range in cells, the distance at which the correlation between two cells falls to 0.05.
model	Correlation model: ‘"exponential"‘, ‘exp(-3 h / range)‘, or ‘"gaussian"‘, ‘exp(-3 (h / range)^2)‘, where ‘h‘ is the distance in cells.

values	Optional numeric vector. If supplied, the simulated field is used only for its spatial ranks and receives the value distribution of 'values': the sorted values when 'length(values)' equals the number of cells, otherwise the corresponding sample quantiles.
res	Cell size in map units.
crs	Coordinate reference system, passed to [terra::rast()].
seed	Optional integer seed. The random-number state of the session is restored on exit.

Details

The covariance is embedded in a torus of twice the grid size in each direction and the field is drawn with two fast Fourier transforms (Wood and Chan 1994; Dietrich and Newsam 1997). Negative eigenvalues of the embedding, which can occur for the Gaussian model at long ranges, are set to zero; the exponential model does not produce them at the ranges used in practice.

Value

A single-layer 'SpatRaster'. Without 'values', the field is standardised to mean 0 and standard deviation 1 over the grid.

References

Wood ATA, Chan G (1994) Simulation of stationary Gaussian processes in $[0,1]^d$. *Journal of Computational and Graphical Statistics* 3:409-432.

Dietrich CR, Newsam GN (1997) Fast and exact simulation of stationary Gaussian processes through circulant embedding of the covariance matrix. *SIAM Journal on Scientific Computing* 18:1088-1107.

Examples

```
z <- ss_simulate(100, 100, range = 10, seed = 1)
z
# the same arrangement with a skewed value distribution
zg <- ss_simulate(100, 100, range = 10, seed = 1,
                  values = stats::rgamma(10000, shape = 2))
```

Index

ss_benchmark, 2
ss_benchmark_combine, 5
ss_decompose, 6
ss_decompose_landscape, 8
ss_extract, 9
ss_metric_list, 11
ss_metrics, 10
ss_perturb, 12
ss_rank_transfer, 14
ss_scale_profile, 15
ss_simulate, 16